GraphOne / FrontierAtlas Intelligence Graph

Architecture & Anti-Bot Strategy — AI Engineer Demo Task

1. Scale Strategy: Collecting 500,000+ Records

The demo pipeline (this submission) proves the pattern at a 1,000-record scale per entity type using real, verifiable sources: Arxiv's API for papers, the YC company directory for startups, and each startup's own website for product/pricing signal. None of this requires code changes to scale — only infrastructure changes:

- **Horizontal worker scaling.** Every scraper here is a single asyncio process with a bounded concurrency semaphore. At 500k scale, the same coroutine logic runs across N worker processes/containers (e.g. Kubernetes Jobs or AWS Fargate tasks), each claiming a partition of the source (a page range, a category, an alphabetical shard of company names) from a shared work queue (SQS/RabbitMQ/Kafka).
- **Source partitioning.** Arxiv has ~30 categories (cs.AI, cs.LG, cs.CL...) that can be crawled in parallel; YC's directory can be sharded by batch/year; job boards can be sharded by role category. Partitioning avoids any single worker becoming a bottleneck or a single point of rate-limiting.
- **Sink, not single-file JSON.** The demo writes one JSON file per run for inspectability. At scale, each worker streams records into a durable sink (Kafka topic → batch loader → Postgres/warehouse) instead of holding everything in memory until the end.

2. Handling 413s and 429s at Scale

Demonstrated in this submission's code (arxiv_scraper.py, llm_extractor.py):

- **429 (rate limited):** every HTTP call goes through a shared retry wrapper that reads `Retry-After` when present, otherwise applies exponential backoff (base × 2^attempt) plus random jitter, capped at a small number of attempts before the call is abandoned rather than retried forever. GitHub's Search API additionally gets its own dedicated semaphore + fixed inter-request delay, because it rate-limits below its documented threshold in practice.
- **413 (payload too large):** the LLM extraction engine treats 413 as a distinct signal, not a generic error — it does not retry the same oversized payload, it calls chunk_text() to split the input on paragraph boundaries into sub-6000-character pieces, extracts each independently, and merges results (see llm_extractor.py: _extract_chunked).
- **At 500k-worker scale:** the same per-provider backoff generalizes to a token-bucket rate limiter shared across workers via Redis (so 50 workers don't each independently retry into the same wall), and chunking parameters become source-aware (e.g. a smaller max-chars budget for verbose HTML pages vs. clean RSS text).

3. Freshness Tracking Across Distributed Nodes

The demo's news_jobs_scraper.py filters strictly on each item's real published_date/date field against a cutoff (now - 24h) — nothing is backfilled or relaxed to pad numbers. At distributed scale, avoiding duplicate processing across nodes needs three additional pieces this demo doesn't need at 1-worker scale:

- **Idempotency key per item** — a hash of (source_url + published_date) written to a shared dedup store (Redis SET or a Bloom filter for cheap approximate membership at high volume) before a worker begins processing it. A second worker that picks up the same item checks the key first and skips.
- **Per-source watermarks** — each worker records the newest published_date it has successfully processed per source. The next poll only requests items after that watermark (where the source API supports it, e.g. Arxiv's sortBy=submittedDate), cutting redundant re-fetching.
- **Continuous polling, not one-shot** — in production this runs on an hourly cron per source rather than a single invocation, so the 24h freshness window is always populated even if any individual run finds nothing new (as several of our 5 news sources correctly did in the demo run).

4. Storage Strategy

Layer	Choice	Why
Primary record store	PostgreSQL	Structured, schema-enforced records (Startup/Product/Paper/Job/News) with strong cons
Relationship / graph layer	Neo4j (or Postgres + Apache AGE)	The product's core value is the “Intelligence Graph” itself — Startup→Product, Paper→G
Semantic search layer	Vector DB (pgvector or Pinecone)	Embeddings of paper abstracts / product descriptions enable similarity search (“find start
Dedup / freshness state	Redis	Sub-millisecond membership checks for the idempotency keys and per-source watermar

5. Anti-Bot & JS-Rendered Source Strategy (Phase V)

This demo's scrapers are all lightweight async HTTP clients (aiohttp) — deliberately, because the sources chosen (Arxiv API, YC's static company mirror, RSS feeds, RemoteOK/Arbeitnow APIs) don't require a browser. For the harder, high-value sources the brief calls out (Cloudflare/Datadome-protected sites, heavily JS-rendered pages like LinkedIn or Crunchbase), the pipeline's strategy is:

- **Playwright (async) as the fallback tier, not the default.** aiohttp is tried first for every URL because it's ~10–20x cheaper in compute and doesn't trip JS-challenge detection at all (there's no JS to fail to execute). Only when a response looks like a challenge page (status 403/503, or a body containing known challenge markers like “checking your browser” / a cf-mitigated header) does the URL get requeued for a Playwright worker pool with a real browser context.
- **Residential/rotating proxy pool** for the Playwright tier specifically — datacenter IPs are the fastest signal Cloudflare/Datadome use to fingerprint scraping traffic; rotating through residential exit nodes per-request (via a provider like Bright Data or Oxylabs) is the single highest-leverage change for this class of site.
- **Human-like pacing and fingerprint variance** — randomized inter-request delay (not a fixed interval), rotating realistic User-Agent + Accept-Language headers, and letting Playwright run with playwright-stealth patches (masks the common headless-browser tells: navigator.webdriver, missing plugins array, etc.).
- **Session persistence over raw retries** — reusing a browser context (and its cookies) across multiple pages of the same domain looks far more like a real user session than a fresh unauthenticated request per page, and avoids re-triggering a challenge on every single request.
- **Respecting the boundary.** Sources that require login (LinkedIn) are treated as out of scope for unauthenticated scraping regardless of technique — the strategy above targets public, technically-protected-but-not-access-controlled pages, not authentication bypass.

Every number quoted elsewhere in this submission (1,000 papers, 1,000 startups, 1,000 products, 397 GitHub-linked papers, 477 priced products, 10 fresh news articles, 85 fresh jobs) came from the pipeline actually run against these real sources — none of it is illustrative or invented.